

CRASH COURSE · 52 PAGES

Flutter AI App Development

A hands-on, project-based path from Flutter fundamentals to shipping real AI features — chat, voice, vision, and retrieval — in production apps.

12 Modules

Flutter 3.x & Dart 3

LLM APIs + On-Device AI

52

PAGES

SKILL LAB
Learning Series

Table of Contents

✓	Welcome & Setup	pp. 3–4	6	State Management	pp. 29–32
1	Flutter Refresher	pp. 5–9	7	On-Device AI	pp. 33–36
2	The AI Landscape	pp. 10–13	8	Voice AI	pp. 37–40
3	Project Setup	pp. 14–17	9	RAG & Vector Search	pp. 41–44
4	Calling LLM APIs	pp. 18–23	10	Multimodal AI	pp. 45–47
5	Building the Chat UI	pp. 24–28	11	Production & Beyond	pp. 48–52

How to read this book: each page is one self-contained lesson — a concept plus working Dart/Flutter code you can copy straight into a project. Work through modules in order, or jump to whichever topic you need right now.

Welcome to the Crash Course

This crash course takes you from a working Flutter developer to someone who can confidently ship AI-powered mobile apps — chatbots, voice assistants, image-understanding features, and retrieval-augmented apps that talk to your own data.

It is organized as 12 modules across 52 pages. Each page is a single, focused lesson: a concept, a pattern, or a piece of working code you can drop into a real project. There is no fluff — every page assumes you have Flutter installed and just want to build.

How this book is structured

- **Modules 1–2** — refresh Flutter fundamentals and map the AI landscape so you pick the right tool for the job.
- **Modules 3–6** — wire up cloud LLM APIs, build a production-grade chat UI, and manage state cleanly.
- **Modules 7–9** — go on-device with TensorFlow Lite and add voice input/output.
- **Modules 10–11** — add retrieval (RAG) and multimodal (image + text) capabilities.
- **Module 12** — testing, cost control, and shipping to the app stores.

How to use this course: read a page, then immediately try the snippet in a scratch Flutter project. Muscle memory beats memorization every time.

Prerequisites & Setup Checklist

You do not need to be a Flutter expert, but you should be comfortable with the basics. Confirm the following before Module 1:

- ☐ Flutter SDK 3.x installed (`flutter doctor` shows no critical errors)
- ☐ A working editor — VS Code or Android Studio with the Flutter/Dart plugins
- ☐ Basic Dart syntax: variables, classes, async/await
- ☐ An emulator or physical device ready to run apps
- ☐ An API key for at least one LLM provider (Anthropic, OpenAI, or Google)
- ☐ Git installed, and a GitHub/GitLab account for version control

Recommended pubspec starter packages

```
dependencies:  
  flutter:  
    sdk: flutter  
  http: ^1.2.0  
  dio: ^5.4.0  
  flutter_dotenv: ^5.1.0  
  provider: ^6.1.2  
  flutter_riverpod: ^2.5.1  
  flutter_markdown: ^0.7.1  
  shared_preferences: ^2.2.2
```

You will meet each of these packages in context throughout the course — no need to install them all now.

Why Flutter for AI Apps

Flutter's single codebase, hot reload, and rich widget system make it an unusually good fit for AI-powered apps, where you are iterating quickly on UI states: loading, streaming tokens, partial results, and errors.

What makes Flutter a strong choice

- **One codebase, every platform** — ship the same AI feature to iOS, Android, web, and desktop.
- **Reactive rendering** — ideal for streaming LLM tokens into a UI in real time.
- **Rich animation system** — typing indicators, shimmer loaders, and smooth transitions are cheap to build.
- **Strong async primitives** — `Future` and `Stream` map naturally onto API calls and streaming responses.
- **Mature package ecosystem** — HTTP clients, ML runtimes, and speech packages are all first-class.

Mental model: think of an AI feature as a state machine — idle → sending → streaming → complete/error. Flutter's widget rebuilds map directly onto that machine.

Dart Essentials You'll Actually Use

A fast refresher on the Dart features that show up constantly when wiring AI into an app.

Null safety & async

```
Future<String> fetchReply(String prompt) async {  
  try {  
    final res = await api.send(prompt);  
    return res.text ?? 'No response';  
  } catch (e) {  
    return 'Error: $e';  
  }  
}
```

Records & pattern matching (Dart 3)

```
(String role, String content) parseMessage(Map<String, dynamic> json) {  
  return (json['role'] as String, json['content'] as String);  
}  
  
final (role, content) = parseMessage(data);
```

Sealed classes for AI response states

```
sealed class ChatState {}  
class Idle extends ChatState {}  
class Streaming extends ChatState { final String partial;  
  Streaming(this.partial); }  
class Done extends ChatState { final String text; Done(this.text); }  
class Failed extends ChatState { final String message; Failed(this.message); }
```

Sealed classes plus a `switch` expression give you exhaustive, compiler-checked UI states — extremely useful once your AI feature has more than two outcomes.

Widgets, State & the Rebuild Cycle

Every AI feature you build is really a widget tree reacting to changing data. Get comfortable with the two core state widgets before adding a state-management library on top.

```
class ChatBubble extends StatelessWidget {
  final String text;
  final bool isUser;
  const ChatBubble({super.key, required this.text, required this.isUser});

  @override
  Widget build(BuildContext context) {
    return Align(
      alignment: isUser ? Alignment.centerRight : Alignment.centerLeft,
      child: Container(
        margin: const EdgeInsets.symmetric(vertical: 4, horizontal: 8),
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: isUser ? const Color(0xFF4F39F6) : const Color(0xFFF1F0FE),
          borderRadius: BorderRadius.circular(16),
        ),
        child: Text(text, style: TextStyle(color: isUser ? Colors.white :
Colors.black)),
      ),
    );
  }
}
```

Key rule

Keep widgets that render AI output *stateless* and push the actual state (messages, streaming buffer, loading flags) up into a [ChangeNotifier](#), Riverpod provider, or Bloc. This keeps rebuilds cheap and predictable even as conversations grow to hundreds of messages.

Layout & Navigation Patterns

Most AI apps share a familiar shell: a scrollable message list, a persistent input bar, and one or more settings/history screens reached by navigation.

```
Scaffold(  
  appBar: AppBar(title: const Text('AI Assistant')),  
  body: Column(  
    children: [  
      Expanded(  
        child: ListView.builder(  
          reverse: true,  
          itemCount: messages.length,  
          itemBuilder: (_, i) => ChatBubble(  
            text: messages[messages.length - 1 - i].content,  
            isUser: messages[messages.length - 1 - i].isUser,  
          ),  
        ),  
      ),  
      const ChatInputBar(),  
    ],  
  ),  
);
```

Navigation for multi-conversation apps

- Use `go_router` for deep-linkable chat threads (`/chat/:threadId`).
- Keep a lightweight `Drawer` or side panel for conversation history on tablet/web layouts.
- Reserve full-screen modals for settings like model choice, temperature, or system prompt.

Async Programming Deep Dive: Future vs Stream

This is the single most important Dart concept for AI apps. A `Future` resolves once; a `Stream` can emit many times — exactly what you need for token-by-token LLM output.

```
// One-shot response
Future<String> getReply(String prompt) async { ... }

// Streamed, token-by-token response
Stream<String> streamReply(String prompt) async* {
  final request = buildRequest(prompt);
  await for (final chunk in httpClient.streamRequest(request)) {
    yield decodeChunk(chunk);
  }
}
```

Consuming a stream in the UI

```
StreamBuilder<String>(
  stream: streamReply(prompt),
  builder: (context, snapshot) {
    final text = snapshot.data ?? '';
    return Text(text);
  },
);
```

Tip: for chat apps, accumulate stream chunks into a mutable buffer inside your state controller rather than rebuilding a raw `StreamBuilder` per message — it scales far better once a conversation has dozens of streamed messages.

Cloud APIs vs. On-Device AI

Before writing code, decide where the intelligence should actually run. This single decision shapes your architecture, cost, and offline behavior.

	Cloud LLM APIs	On-Device Models
Capability	State-of-the-art reasoning, huge context	Narrower tasks: classification, detection
Latency	Network-dependent	Near-instant, no round trip
Cost	Per-token, scales with usage	One-time model size, free inference
Privacy	Data leaves the device	Data stays on-device
Offline	Requires connectivity	Works fully offline

Most production apps use **both**: an on-device model for fast, cheap, always-available tasks (e.g. blurring a background, flagging offensive text) and a cloud LLM for open-ended reasoning and conversation.

A Tour of LLM Providers

You'll typically talk to these APIs over plain HTTPS with JSON — no special SDK is required, though official and community Dart packages exist for all three.

- **Anthropic Claude** — `POST https://api.anthropic.com/v1/messages`. Strong at long-context reasoning, tool use, and following detailed system prompts.
- **OpenAI** — `POST https://api.openai.com/v1/chat/completions` (or the newer Responses API). Broad ecosystem, function calling, and vision support.
- **Google Gemini** — `POST https://generativelanguage.googleapis.com/v1/models/...`. Tight integration with Firebase and native multimodal input.

What they have in common

- A `messages` array of `{role, content}` objects.
- A streaming mode using Server-Sent Events (SSE).
- Token-based pricing and rate limits you must design around.

This course uses Anthropic's Claude API in examples, but every pattern here ports directly to OpenAI or Gemini — only the request/response shape changes.

On-Device ML Options for Flutter

When you need speed, offline support, or to avoid sending data off-device, reach for one of these.

- **Google ML Kit** — ready-made APIs for text recognition, face detection, barcode scanning, and language identification. Easiest starting point, via `google_mlkit_*` packages.
- **TensorFlow Lite** (`tflite_flutter`) — run your own trained or converted `.tflite` models for custom classification, detection, or embedding generation.
- **Core ML** (iOS-only, via platform channels) — Apple's native runtime, useful when you already have a `.mlmodel` and only target iOS.

Rule of thumb: start with ML Kit for common tasks. Drop to raw TFLite only when you need a custom-trained model ML Kit doesn't cover.

Choosing the Right Approach for Your App

Use this decision guide when scoping a new AI feature.

Open-ended chat, summarization, writing help	→ Cloud LLM
Classify a photo into known categories	→ On-device (ML Kit / TFLite)
Search your own documents by meaning	→ Cloud embeddings + vector search (RAG)
Transcribe speech to text	→ On-device STT, or cloud for higher accuracy
Understand an uploaded image in conversation	→ Cloud multimodal LLM
Real-time camera effects (no network)	→ On-device TFLite model

Keep this table nearby — nearly every architecture decision in this course traces back to one of these rows.

Scaffolding an AI-Ready Flutter Project

Start with a folder structure that separates AI plumbing from UI, so you can swap providers or add features without a rewrite.

```
lib/  
  main.dart  
  core/  
    config/          # env, constants  
    network/         # http client, interceptors  
  features/  
    chat/  
      data/          # API clients, models  
      state/         # providers / blocs  
      widgets/       # chat bubble, input bar  
    vision/  
    voice/  
  shared/  
    theme/  
    widgets/
```

```
flutter create ai_crash_course  
cd ai_crash_course  
flutter pub add http dio flutter_dotenv provider flutter_riverpod
```

This layered structure means `features/chat/data` is the only place that knows which LLM provider you're using — everything above it just consumes a clean Dart interface.

Essential Packages, Explained

<code>http</code>	Lightweight HTTP client — perfect for simple request/response API calls.
<code>dio</code>	Feature-rich client with interceptors, retries, and easier streaming — use for production apps.
<code>flutter_dotenv</code>	Loads API keys from a local <code>.env</code> file, kept out of version control.
<code>provider</code> / <code>flutter_riverpod</code>	State management for chat/session state.
<code>flutter_markdown</code>	Renders LLM output that includes Markdown formatting.
<code>shared_preferences</code>	Simple local persistence for chat history and settings.
<code>tflite_flutter</code>	Runs on-device TensorFlow Lite models.
<code>speech_to_text</code> / <code>flutter_tts</code>	Voice input and output.

Securing Your API Keys

Never hard-code an API key into your Dart source — it ships inside the compiled app and can be extracted.

Local development: .env files

```
# .env (add this file to .gitignore!)  
ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxx
```

```
await dotenv.load(fileName: '.env');  
final apiKey = dotenv.env['ANTHROPIC_API_KEY']!;
```

Production: never call the LLM directly from the client

For a shipped app, put your API key on a thin backend (a Cloud Function, Firebase callable function, or small server) and have Flutter call *that*. Your key never touches the device.

```
Client (Flutter) --> Your backend (holds the key) --> LLM provider
```

Non-negotiable: an API key embedded in a published app is a leaked API key. Treat this proxy step as mandatory before release, not an optimization.

Environment Configuration & Flavors

Most AI apps need at least two configurations: development (talking to a test backend / cheap model) and production (talking to your real backend / production model).

```
abstract class Env {  
  static const String apiUrl = String.fromEnvironment(  
    'API_BASE_URL',  
    defaultValue: 'https://dev.api.yourapp.com',  
  );  
  static const String model = String.fromEnvironment(  
    'MODEL_NAME',  
    defaultValue: 'claude-sonnet-4-6',  
  );  
}
```

```
flutter run --dart-define=API_BASE_URL=https://prod.api.yourapp.com \  
  --dart-define=MODEL_NAME=claude-opus-5
```

Wire these into Flutter's build flavors (or a simple `--dart-define` flag as above) so switching between a cheap/fast model in development and your production model is a one-line change, not a code edit.

Your First API Call

A minimal, non-streaming call to an LLM chat endpoint using the `http` package.

```
Future<String> askAI(String prompt) async {
  final response = await http.post(
    Uri.parse('https://your-backend.com/api/chat'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({
      'messages': [
        {'role': 'user', 'content': prompt}
      ],
      'model': 'claude-sonnet-4-6',
      'max_tokens': 1024,
    }),
  );

  if (response.statusCode != 200) {
    throw Exception('API error: ${response.statusCode}');
  }
  final data = jsonDecode(response.body);
  return data['content'][0]['text'] as String;
}
```

Notice this calls *your backend*, not the provider directly — consistent with the key-security pattern from Module 3.

Modeling Requests & Responses

Wrap raw JSON in typed Dart classes early — it pays off the moment you add streaming, tool use, or multimodal content.

```
class ChatMessage {
  final String role; // 'user' | 'assistant' | 'system'
  final String content;
  ChatMessage({required this.role, required this.content});

  Map<String, dynamic> toJson() => {'role': role, 'content': content};
  factory ChatMessage.fromJson(Map<String, dynamic> json) =>
    ChatMessage(role: json['role'], content: json['content']);
}

class ChatResponse {
  final String text;
  final int inputTokens;
  final int outputTokens;
  ChatResponse({required this.text, required this.inputTokens, required
    this.outputTokens});
}
```

Keep these models in `features/chat/data/models/` and have your API client return them directly, so nothing above the data layer touches raw JSON.

Robust Error Handling & Retries

Network calls to an LLM fail in predictable ways: rate limits, timeouts, and transient 5xx errors. Handle each distinctly rather than showing a generic "Something went wrong."

```
Future<ChatResponse> sendWithRetry(List<ChatMessage> messages, {int retries = 3})
  async {
    for (var attempt = 0; attempt < retries; attempt++) {
      try {
        return await _send(messages);
      } on RateLimitException {
        await Future.delayed(Duration(seconds: pow(2, attempt).toInt()));
      } on TimeoutException {
        if (attempt == retries - 1) rethrow;
      } on ApiException catch (e) {
        if (e.statusCode >= 500 && attempt < retries - 1) continue;
        rethrow; // 4xx errors (bad request, auth) should not retry
      }
    }
    throw Exception('Max retries exceeded');
  }
```

UX matters here: surface "Retrying..." for transient errors, but fail fast and clearly for auth or bad-request errors — retrying those just wastes the user's time.

Streaming Responses with Server-Sent Events

Streaming is what makes an AI chat feel alive — text appears as it's generated instead of after a long pause. Most providers stream via SSE over a chunked HTTP response.

```
Stream<String> streamChat(List<ChatMessage> messages) async* {
  final request = http.Request('POST', Uri.parse('$baseUrl/chat/stream'))
    ..headers['Content-Type'] = 'application/json'
    ..body = jsonEncode({'messages': messages.map((m) => m.toJson()).toList(),
    'stream': true});

  final streamed = await request.send();
  final lines = streamed.stream.transform(utf8.decoder).transform(const
  LineSplitter());

  await for (final line in lines) {
    if (!line.startsWith('data: ')) continue;
    final payload = line.substring(6);
    if (payload == '[DONE]') break;
    final json = jsonDecode(payload);
    final delta = json['delta']?['text'];
    if (delta != null) yield delta as String;
  }
}
```

Accumulate each `yield` ed chunk into your message's text as it arrives — covered in Module 6.

Building the Chat Data Model

A conversation is just an ordered list of messages plus some metadata. Model it once and reuse it everywhere: persistence, the UI, and the API payload.

```
class Conversation {  
  final String id;  
  final List<ChatMessage> messages;  
  final DateTime updatedAt;  
  
  Conversation({required this.id, required this.messages, required  
    this.updatedAt});  
  
  Conversation appending(ChatMessage message) => Conversation(  
    id: id,  
    messages: [...messages, message],  
    updatedAt: DateTime.now(),  
  );  
}
```

Because `Conversation` is immutable, appending a message returns a new instance — ideal for state-management tools like Riverpod or Bloc that key off object identity to decide when to rebuild.

Rate Limiting & Cost Management

LLM calls cost real money per token, and providers enforce request-per-minute limits. Design for both from day one.

- **Debounce sends** — disable the send button while a request is in flight; never allow duplicate submissions.
- **Cap context length** — truncate or summarize old messages before they blow your token budget on every request.
- **Set `max_tokens` deliberately** — an unbounded response can silently spike cost.
- **Track usage server-side** — log `input_tokens` / `output_tokens` per user so you can set fair-use caps.
- **Cache repeat queries** — identical prompts (e.g. FAQ-style) don't need to hit the model twice.

Ship a budget guardrail: a simple per-user daily token cap, enforced on your backend, prevents a single runaway client from generating a surprise bill.

Designing the Chat Screen Layout

A chat screen has three zones: a scrollable history, an input bar pinned to the bottom, and optional context chrome (model picker, thread title). Get the skeleton right before styling.

```
Scaffold(  
  appBar: AppBar(title: Text(conversation.title)),  
  body: SafeArea(  
    child: Column(  
      children: [  
        Expanded(child: MessageList(messages: conversation.messages)),  
        const Divider(height: 1),  
        ChatInputBar(onSend: controller.send),  
      ],  
    ),  
  ),  
);
```

Wrap the input bar with `SafeArea` and handle keyboard insets so the input never gets covered on smaller devices — use a `Padding` bound to `MediaQuery.of(context).viewInsets.bottom`.

Message Bubbles with ListView.builder

Never build a chat list with a fixed `Column` — use `ListView.builder` so only visible bubbles are laid out, which matters once a conversation reaches hundreds of turns.

```
ListView.builder(  
  reverse: true,  
  padding: const EdgeInsets.symmetric(vertical: 8),  
  itemCount: messages.length,  
  itemBuilder: (context, index) {  
    final message = messages[messages.length - 1 - index];  
    return ChatBubble(message: message);  
  },  
);
```

`reverse: true` keeps the list pinned to the latest message and naturally handles auto-scroll as new tokens stream in — no manual `ScrollController` gymnastics required for the common case.

Typing Indicators & Loading States

A visible "thinking" state builds trust that the app is working, not frozen — especially important during the gap before the first streamed token arrives.

```
class TypingIndicator extends StatelessWidget {
  const TypingIndicator({super.key});
  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: List.generate(3, (i) => Padding(
        padding: const EdgeInsets.symmetric(horizontal: 2),
        child: _Dot(delay: i * 150),
      )),
    );
  }
}
```

Drive each dot's opacity with a staggered [AnimationController](#) loop. Show this widget the moment a request is sent, and swap it for real text the instant the first stream chunk arrives.

Rendering Markdown in Chat

LLMs commonly reply with Markdown — headings, code fences, bullet lists. Render it properly instead of showing raw asterisks and backticks.

```
MarkdownBody(  
  data: message.content,  
  selectable: true,  
  styleSheet: MarkdownStyleSheet(  
    code: const TextStyle(fontFamily: 'monospace', backgroundColor:  
Color(0xFFFF1F0FE)),  
    codeblockDecoration: BoxDecoration(  
      color: const Color(0xFF09090B),  
      borderRadius: BorderRadius.circular(8),  
    ),  
  ),  
);
```

Streaming caveat: partial Markdown (an unclosed code fence mid-stream) can render oddly for a split second. It's usually fine to accept this flicker; if not, buffer until a fence closes before rendering that block.

Persisting Chat History Locally

Users expect conversations to survive an app restart. For most apps, local persistence via `shared_preferences` or a lightweight database like `sqflite` / `Hive` is enough.

```
Future<void> saveConversation(Conversation conversation) async {
  final prefs = await SharedPreferences.getInstance();
  final json = jsonEncode({
    'id': conversation.id,
    'messages': conversation.messages.map((m) => m.toJson()).toList(),
  });
  await prefs.setString('conversation_${conversation.id}', json);
}
```

For apps with many long conversations, prefer `Hive` or `sqflite` over `SharedPreferences` — they scale far better than storing large JSON blobs in key-value storage.

Why State Management Matters Here

An AI chat feature has more moving state than a typical CRUD screen: the message list, a streaming buffer, loading/error flags, the selected model, and possibly multiple concurrent conversations. Ad hoc `setState` calls get unmanageable fast.

What you actually need to track

- Current conversation and its message list
- In-flight streaming text (before it's committed as a full message)
- Request status: idle / sending / streaming / error
- Model & settings (temperature, system prompt) selected by the user

The next three lessons implement the same chat controller three ways — Provider, Riverpod, and Bloc — so you can pick the pattern that fits your team.

Provider Walkthrough

Provider (specifically **ChangeNotifier**) is the simplest starting point.

```
class ChatController extends ChangeNotifier {
  final List<ChatMessage> messages = [];
  bool isStreaming = false;

  Future<void> send(String text) async {
    messages.add(ChatMessage(role: 'user', content: text));
    isStreaming = true;
    notifyListeners();

    final buffer = StringBuffer();
    await for (final chunk in api.streamChat(messages)) {
      buffer.write(chunk);
      notifyListeners(); // rebuild on every chunk
    }
    messages.add(ChatMessage(role: 'assistant', content: buffer.toString()));
    isStreaming = false;
    notifyListeners();
  }
}
```

```
ChangeNotifierProvider(
  create: (_) => ChatController(),
  child: const ChatScreen(),
);
```

Riverpod for AI State

Riverpod adds compile-time safety and easy testability over Provider, and shines when you have several interdependent pieces of AI state (model choice, streaming buffer, conversation list).

```
final chatProvider = StateNotifierProvider<ChatNotifier, ChatState>(
  (ref) => ChatNotifier(ref.read(apiClientProvider)),
);

class ChatNotifier extends StateNotifier<ChatState> {
  final ApiClient _api;
  ChatNotifier(this._api) : super(ChatState.idle());

  Future<void> send(String text) async {
    state = state.copyWith(status: Status.streaming);
    final buffer = StringBuffer();
    await for (final chunk in _api.streamChat(state.messages, text)) {
      buffer.write(chunk);
      state = state.copyWith(partial: buffer.toString());
    }
    state = state.commitPartial();
  }
}
```

```
final chat = ref.watch(chatProvider);
Text(chat.partial.isNotEmpty ? chat.partial : chat.lastMessage);
```

Bloc for Complex AI Flows

Reach for Bloc when an AI feature has many discrete events (send, cancel, retry, switch model, regenerate) and you want a strict, auditable event-to-state pipeline — common in larger teams.

```
sealed class ChatEvent {}
class SendMessage extends ChatEvent { final String text; SendMessage(this.text); }
class CancelStream extends ChatEvent {}
class RegenerateLast extends ChatEvent {}

class ChatBloc extends Bloc<ChatEvent, ChatState> {
  ChatBloc(this._api) : super(ChatState.idle()) {
    on<SendMessage>(_onSend);
    on<CancelStream>((event, emit) => _subscription?.cancel());
  }

  final ApiClient _api;
  StreamSubscription? _subscription;

  Future<void> _onSend(SendMessage event, Emitter<ChatState> emit) async {
    emit(state.copyWith(status: Status.streaming));
    await emit.forEach(_api.streamChat(state.messages, event.text),
      onData: (chunk) => state.copyWith(partial: state.partial + chunk));
  }
}
```


Intro to TensorFlow Lite in Flutter

TensorFlow Lite (TFLite) runs a pre-trained, compressed model directly on the device — no network call, no per-inference cost, and it works offline.

```
flutter pub add tflite_flutter
```

```
import 'package:tflite_flutter/tflite_flutter.dart';

class ImageClassifier {
  late final Interpreter _interpreter;

  Future<void> load() async {
    _interpreter = await Interpreter.fromAsset('assets/model.tflite');
  }

  List<double> classify(List<List<List<double>>> input) {
    final output = List.filled(1 * 1000, 0.0).reshape([1, 1000]);
    _interpreter.run(input, output);
    return output[0];
  }
}
```

Remember to declare your `.tflite` file under `assets:` in `pubspec.yaml` or it won't be bundled into the app.

Building an Image Classification Feature

A common pattern: pick or capture an image, preprocess it into the tensor shape your model expects, run inference, then map the output vector to a human-readable label.

```
Future<String> classifyImage(File imageFile) async {  
  final bytes = await imageFile.readAsBytes();  
  final image = img.decodeImage(bytes!);  
  final resized = img.copyResize(image, width: 224, height: 224);  
  
  final input = _imageToTensor(resized); // normalize 0-255 -> 0-1  
  final scores = classifier.classify(input);  
  
  final topIndex = scores.indexWhere((s) => s == scores.reduce(max));  
  return labels[topIndex];  
}
```

Common pitfall: mismatched input size or normalization between how the model was trained and how you preprocess in Dart is the #1 cause of nonsensical predictions. Always confirm the expected input shape with `interpreter.getInputTensor(0).shape`.

Real-Time Object Detection

For live camera object detection, combine the `camera` package's image stream with a detection model (e.g. a MobileNet-SSD `.tflite` file), running inference on a background isolate to keep the UI smooth.

```
CameraController controller = CameraController(camera, ResolutionPreset.medium);
await controller.initialize();

controller.startImageStream((CameraImage frame) {
  if (_isProcessing) return; // drop frames while busy
  _isProcessing = true;
  _detector.detect(frame).then((results) {
    setState(() => _boxes = results);
    _isProcessing = false;
  });
});
```

Draw the resulting bounding boxes with a `CustomPainter` layered over the camera preview using a `Stack`.

Performance: Keeping On-Device AI Smooth

- **Run inference off the UI thread** — use `compute()` or an `Isolate` for anything heavier than a few milliseconds.
- **Quantize your model** — an INT8-quantized `.tflite` model can be 4x smaller and significantly faster than float32, with a small accuracy trade-off.
- **Throttle frame processing** — for live camera streams, drop frames while a previous inference is still running rather than queuing them.
- **Reuse the interpreter** — load the model once at app start, not per-inference.
- **Profile on real devices** — emulator performance is not representative; always test on a mid-range physical phone.

Speech-to-Text Integration

The `speech_to_text` package wraps each platform's native recognizer — no server round trip needed for basic transcription.

```
flutter pub add speech_to_text
```

```
final speech = SpeechToText();

Future<void> startListening(void Function(String) onResult) async {
  final available = await speech.initialize();
  if (!available) return;
  speech.listen(
    onResult: (result) => onResult(result.recognizedWords),
    listenFor: const Duration(seconds: 30),
  );
}
```

Remember to request the microphone permission (via `permission_handler` or platform manifest entries) before calling `initialize()`, or it will silently fail on first launch.

Text-to-Speech Output

`flutter_tts` speaks an LLM's text response aloud — useful for hands-free assistants and accessibility.

```
flutter pub add flutter_tts
```

```
final tts = FlutterTts();

Future<void> speak(String text) async {
  await tts.setLanguage('en-US');
  await tts.setSpeechRate(0.5);
  await tts.speak(text);
}
```

Speaking streamed text

For a streaming LLM response, queue TTS by sentence rather than waiting for the full reply — split on sentence-ending punctuation and call `tts.speak()` per completed sentence for a natural, low-latency voice experience.

Building a Voice Assistant Flow

A complete voice loop chains four steps: listen → transcribe → send to the LLM → speak the reply.

```
Future<void> voiceTurn() async {  
  setState(() => status = VoiceStatus.listening);  
  final transcript = await listenOnce();  
  
  setState(() => status = VoiceStatus.thinking);  
  final reply = await askAI(transcript);  
  
  setState(() => status = VoiceStatus.speaking);  
  await speak(reply);  
  
  setState(() => status = VoiceStatus.idle);  
}
```

Drive a single animated mic/waveform widget off the `VoiceStatus` enum so the user always has a clear signal of which phase the assistant is in — ambiguity here is the most common voice-UX complaint.

Combining Voice with a Streaming LLM

For the lowest-latency voice assistant, don't wait for the full LLM response before speaking — start speaking the first sentence while later sentences are still streaming in.

```
Future<void> speakStreamed(Stream<String> textStream) async {
  final buffer = StringBuffer();
  await for (final chunk in textStream) {
    buffer.write(chunk);
    final text = buffer.toString();
    final sentenceEnd = RegExp(r'[.!?]\s').firstMatch(text);
    if (sentenceEnd != null) {
      final sentence = text.substring(0, sentenceEnd.end).trim();
      await tts.speak(sentence);
      buffer.clear();
      buffer.write(text.substring(sentenceEnd.end));
    }
  }
  if (buffer.isNotEmpty) await tts.speak(buffer.toString());
}
```


What Is RAG and Why It Matters

Retrieval-Augmented Generation (RAG) lets an LLM answer questions about content it was never trained on — your app's documents, a knowledge base, or user-uploaded files — by retrieving relevant passages and inserting them into the prompt.

The RAG pipeline, at a glance

1. Split your source documents into small chunks.
2. Convert each chunk into a numeric **embedding** vector.
3. Store the vectors in a searchable index.
4. At query time, embed the user's question and retrieve the most similar chunks.
5. Insert those chunks into the LLM prompt as context before asking it to answer.

Why it beats fine-tuning for most apps: RAG lets you update the knowledge base instantly (just re-index) without retraining a model, and it lets the LLM cite exactly which source it used.

Understanding Embeddings

An embedding is a list of floating-point numbers that represents the *meaning* of a piece of text. Texts with similar meaning end up with vectors that are close together in that space.

```
Future<List<double>> embed(String text) async {
  final response = await http.post(
    Uri.parse('$baseUrl/embeddings'),
    body: jsonEncode({'input': text, 'model': 'text-embedding-3-small'}),
  );
  final data = jsonDecode(response.body);
  return List<double>.from(data['embedding']);
}
```

Measuring similarity: cosine similarity

```
double cosineSimilarity(List<double> a, List<double> b) {
  var dot = 0.0, normA = 0.0, normB = 0.0;
  for (var i = 0; i < a.length; i++) {
    dot += a[i] * b[i];
    normA += a[i] * a[i];
    normB += b[i] * b[i];
  }
  return dot / (sqrt(normA) * sqrt(normB));
}
```

A Simple Vector Search Implementation

For small-to-medium knowledge bases (a few thousand chunks), you don't need a dedicated vector database — a simple in-memory or on-device linear scan works fine.

```
class VectorStore {
  final List<(String text, List<double> embedding)> _items = [];

  void add(String text, List<double> embedding) => _items.add((text, embedding));

  List<String> search(List<double> queryEmbedding, {int topK = 4}) {
    final scored = _items
      .map((item) => (item.$1, cosineSimilarity(item.$2, queryEmbedding)))
      .toList()
      ..sort((a, b) => b.$2.compareTo(a.$2));
    return scored.take(topK).map((e) => e.$1).toList();
  }
}
```

Beyond a few thousand chunks, move this store server-side and use a purpose-built vector database (Pinecone, Weaviate, pgvector) for speed and scale.

Wiring RAG into Your Chat App

Bring embeddings, retrieval, and the LLM call together into a single answer function.

```
Future<String> answerWithContext(String question, VectorStore store) async {  
  final queryEmbedding = await embed(question);  
  final relevantChunks = store.search(queryEmbedding, topK: 4);  
  
  final context = relevantChunks.join('\n---\n');  
  final prompt = '''  
Use only the following context to answer the question.  
If the answer isn't in the context, say you don't know.  
  
Context:  
$context  
  
Question: $question  
''';  
  
  return askAI(prompt);  
}
```

Grounding matters: explicitly instructing the model to say "I don't know" when context is insufficient meaningfully reduces hallucinated answers.

Picking Images & Calling a Vision API

Modern LLMs can accept images alongside text. Capture or pick an image, base64-encode it, and include it in the message content array.

```
flutter pub add image_picker
```

```
final picker = ImagePicker();
final file = await picker.pickImage(source: ImageSource.gallery);
final bytes = await file!.readAsBytes();
final base64Image = base64Encode(bytes);

final payload = {
  'messages': [
    {
      'role': 'user',
      'content': [
        {'type': 'text', 'text': 'What is in this image?'},
        {'type': 'image', 'source': {'type': 'base64', 'media_type': 'image/jpeg',
'data': base64Image}},
      ],
    },
  ],
};
```

Building an Image Analysis Feature

A typical flow: capture or select an image, show a preview with a loading overlay, send it for analysis, and render the structured result.

```
class ImageAnalysisScreen extends StatefulWidget { ... }

Future<void> _analyze(File image) async {
  setState(() => _isLoading = true);
  final description = await visionApi.analyze(image);
  setState(() {
    _result = description;
    _isLoading = false;
  });
}
```

For structured extraction (e.g. "list every item and its price"), ask the model to respond in strict JSON and parse it with a typed model — the same pattern from Module 4 — rather than parsing free text.

Multimodal Chat: Text + Image Together

Let users attach an image to any chat turn, not just a dedicated "analyze image" screen, by extending your message model to hold optional attachments.

```
class ChatMessage {  
  final String role;  
  final String content;  
  final List<String> imageBase64;  
  ChatMessage({required this.role, required this.content, this.imageBase64 = const  
    []});  
}
```

In the input bar, add an image-attach button next to the text field; render a thumbnail preview above the input before sending. On the wire, expand the message into the provider's multi-part content array exactly as shown in Lesson 1 of this module.

Testing AI Features

LLM output is non-deterministic, so you can't unit-test exact response text — but you can and should test everything around it.

- **Unit test** your JSON parsing, retry logic, and data models with fixed fixture responses.
- **Widget test** the chat UI by injecting a fake API client that returns canned responses.
- **Golden test** chat bubbles, markdown rendering, and loading states for visual regressions.
- **Integration test** the full send → stream → render flow against a mock server.

```
test('parses streamed chunks into full message', () async {  
  final fakeApi = FakeApiClient(chunks: ['Hel', 'lo', ' world']);  
  final controller = ChatController(fakeApi);  
  await controller.send('hi');  
  expect(controller.messages.last.content, 'Hello world');  
});
```


Designing Graceful Error & Fallback States

Every AI feature will occasionally fail — a timeout, a content filter rejection, a rate limit. Design each of these explicitly rather than letting an exception bubble into a generic crash screen.

Network/ timeout error	Show a retry button inline in the chat, not a full-screen error
Rate limit hit	"You're sending messages quickly — try again in a moment"
Content filtered	Explain briefly, don't expose raw provider error text to the user
Empty/ short response	Offer a "regenerate" action
Fully offline	Fall back to an on-device model or a clear offline message

Performance & Cost Optimization Checklist

- ☐ Streaming enabled for all chat responses (perceived latency)
 - ☐ `max_tokens` set deliberately, not left unbounded
 - ☐ Old conversation context trimmed or summarized before each request
 - ☐ Per-user rate limiting and daily token caps enforced server-side
 - ☐ Repeated/common queries cached where appropriate
 - ☐ On-device models used for simple, high-frequency tasks
 - ☐ Images resized/compressed before sending to a vision API
 - ☐ List views use `.builder` constructors everywhere
-

Deployment Checklist: iOS, Android & Web

☐ API keys removed from the client; all AI calls routed through your backend

☐ Production `--dart-define` flavor configured and verified

☐ Microphone/camera/photo-library permission strings set in `Info.plist` / `AndroidManifest.xml`

☐ Error states tested with airplane mode and a throttled network

☐ App Store / Play Store privacy disclosures updated for AI data usage

☐ Rate limiting and cost alerts live on the backend before public launch

☐ Crash reporting (e.g. Sentry/Crashlytics) wired into the AI request path

Capstone Project & Where to Go Next

Capstone: build a personal knowledge assistant

Combine everything from this course into one app:

- A chat screen with streaming responses (Modules 4–6)
- RAG over a small set of your own notes or PDFs (Module 9)
- An optional voice input/output mode (Module 8)
- Image upload so you can ask questions about a photo of a document (Module 10)
- Proper error states, a backend proxy for your API key, and a cost cap (Module 11)

Where to go from here

- Explore function/tool calling to let the model take actions inside your app.
- Add multi-agent workflows for more complex tasks.
- Benchmark different models for cost vs. quality on your specific use case.
- Study production RAG systems for chunking strategy and re-ranking.

You're ready. Every pattern here — streaming, state, retrieval, voice, vision — is a building block. Combine them deliberately, and ship something real.